

CAT-SOOP Learning Analytics and Instructor Dashboard

Project Plan and Requirements Document

Software Tools and Techniques for CSE • IIT Gandhinagar

Sponsoring Faculty: Prof. Manu Awasthi

Target Deployment: Digital Systems course, CAT-SOOP LMS

Document Status: v1.0 — Initial Project Plan

Executive Summary

Prof. Manu Awasthi runs the Digital Systems course at IIT Gandhinagar (~250 students) on **CAT-SOOP**, an open-source, Python-based learning management system originally built at MIT for automated collection and assessment of exercises. CAT-SOOP already handles course content delivery and auto-grading; Google Classroom is used only as a drop box for YouTube links to circuit-demonstration videos. Following discussion with the professor, **account-provisioning automation has been de-prioritized**, and the project's core deliverable is now an **instructor-facing analytics dashboard** built on top of CAT-SOOP's existing data.

CAT-SOOP does not store data in a conventional application database (no PostgreSQL/MySQL by default). Its own documentation states that *“all of CAT-SOOP's data are stored in files on disk in a directory called `_logs`”* under the configured data root. The project therefore treats **data extraction from this file-based log store** as the first and most important technical workstream, ahead of any dashboard or database design. This plan explicitly separates what has been **confirmed from CAT-SOOP's own source history and documentation** from what **needs to be verified experimentally** against a local test deployment, per the professor's instruction.

The system will be developed and validated first on a local CAT-SOOP instance with a dummy course and synthetic student personas, then adapted to extract data from the professor's real deployment. The architecture is designed to scale beyond the current 250-student class in anticipation of the course being made public.

At a Glance

Item	Summary
Problem	CAT-SOOP handles content delivery and auto-grading well, but exposes no instructor-facing analytics: no view of module reach/completion, attempt patterns, score distributions, difficulty, or at-risk students.
Proposed Solution	A data-extraction and analytics layer built on top of CAT-SOOP's native file-based log store, feeding a purpose-built analytics database and an instructor dashboard, refreshed in daily batches.
Inputs	CAT-SOOP course structure, user records, per-page/per-question log entries (attempts, scores, timestamps), checker results. Google Classroom video-submission data is an optional, later-phase input.
Outputs	Course-, module-, question-, and student-level analytics pages: completion, scores, attempt counts, difficulty indicators, progress-over-time, and an at-risk-student view.
Core Technologies	Python (matches CAT-SOOP itself) for extraction/ETL; a relational analytics store (exact engine chosen after investigating CAT-SOOP's own log format); FastAPI for the analytics API; React + a charting library for the dashboard.
Development Methodology	Iterative/incremental, ten phases, starting with CAT-SOOP source investigation and a local test deployment before any dashboard code is written.
Key Deliverables	Local CAT-SOOP test environment with synthetic students; a data-exploration report; an extraction module; an analytics schema and pipeline; an MVP dashboard; a validated extension roadmap.
Most Important Technical Unknown	Whether CAT-SOOP's log entries give a reliable, page-level signal for “a student reached module X”, and whether the IITGN deployment's on-disk log format/version matches what we validate locally.

1. Project Overview

CAT-SOOP is a free/libre (AGPLv3), Python-based learning management system originally developed at MIT for the 6.01/6.02 Introduction to Electrical Engineering and Computer Science sequence. It is designed to be programmable: course content, exercises, and auto-graders are written as Python-backed content files rather than configured through a conventional admin UI. It continues to be used for several current MIT courses (e.g., 6.100A/6.100L, 6.100B), and remains actively maintained, with its canonical source now hosted on a self-run Gitea/Forgejo instance at catsoop.org/git rather than on GitHub.

In the Digital Systems course at IIT Gandhinagar, CAT-SOOP is the primary platform: students read module content, attempt embedded exercises and quizzes, and receive automatic grading through CAT-SOOP's checker mechanism. Separately, students record a video demonstrating and explaining their circuit and submit the YouTube link through Google Classroom — Google Classroom is not used for course content or grading.

The current limitation is not in CAT-SOOP's ability to deliver content or grade exercises — that part already works. The limitation is that CAT-SOOP, in the deployment the professor uses, does not expose a course-analytics layer: there is no dashboard for seeing how many students have reached or completed a given module, how difficult a given exercise is in aggregate, which students are falling behind, or how the class is progressing as a whole. That instructor-facing analytics capability is what this project builds.

2. Problem Statement

Raw activity and assessment data already exists somewhere inside CAT-SOOP for every student action — every exercise attempt, every checker run, every score. What is missing is a **meaningful, aggregated, instructor-facing view** of that data at three levels:

- **Course level:** overall progress, completion, and assessment performance across the whole class.
- **Module level:** how many students have reached and completed each module, how difficult each module appears to be, and where students are dropping off.
- **Student level:** individual progress, scores, attempt patterns, and whether a given student needs attention.

The professor has no convenient way to answer questions such as *“what percentage of students have reached Module 5?”* or *“which assessment has the highest failure rate?”* without manually inspecting CAT-SOOP's underlying storage. This project treats that gap — not the LMS functionality itself — as the problem to be solved.

3. Stakeholders and Users

Stakeholder	Role / Need
Instructor / Admin (Prof. Manu Awasthi)	Primary user of the analytics dashboard. Needs course-, module-, and student-level views to guide teaching decisions, identify struggling students, and evaluate module/question difficulty.
Students (USER role in CAT-SOOP)	Not direct users of this system. Their activity is the data source. Their data must be handled with appropriate access control (Section 19).

Stakeholder	Role / Need
Guest users	CAT-SOOP's guest role can view course content without full student capabilities; guest activity, if any, is out of scope for per-student analytics.
Analytics system (this project)	Extracts, normalizes, stores, and presents data derived from CAT-SOOP. Runs as a separate service alongside the CAT-SOOP installation, not as a modification to CAT-SOOP's core request path.
CAT-SOOP installation	The source of truth for all course, user, and activity data. Treated as an external system whose on-disk format we read, not one we modify, for the initial phases of the project.

4. Current System Workflow

4.1 Content and Assessment Path (CAT-SOOP)

```

Student
|
v
CAT-SOOP course page
|
v
Reads module content
|
v
Attempts embedded exercises / quizzes
|
v
CAT-SOOP checker runs, auto-grades
|
v
Result recorded in CAT-SOOP's file-based log store
(exact record shape: see Section 12, needs local verification)

```

4.2 Video Submission Path (Google Classroom)

```

Student
|
v
Records circuit demonstration + explanation video
|
v
Uploads to YouTube, submits link
|
v
Google Classroom (used only as a submission drop-box)

```

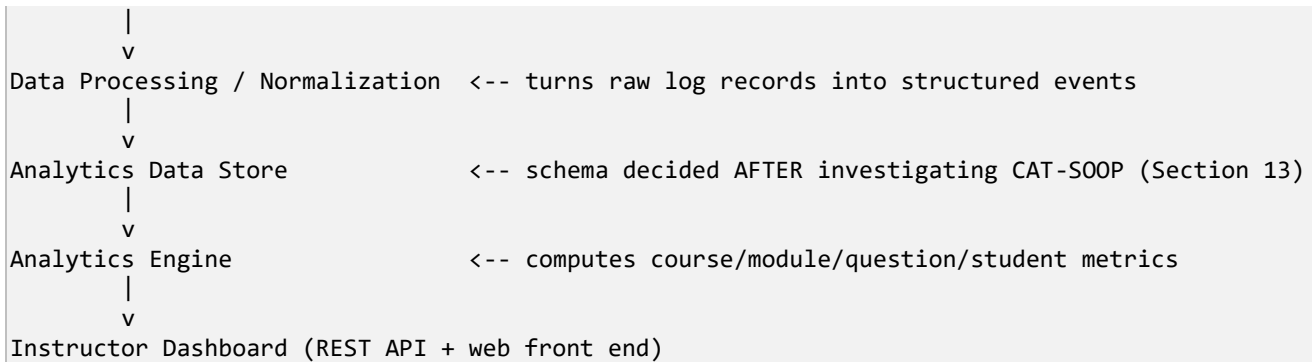
Google Classroom is outside the scope of the analytics dashboard for the initial phases of this project (see Section 17). It is retained only as a possible future data source for the video-submission component.

5. Proposed System

```

CAT-SOOP (file-based log store, cs_data_root/_logs)
|
v
Data Extraction / Access Layer <-- reads CAT-SOOP's on-disk records

```



Each layer is deliberately decoupled from CAT-SOOP's request-handling path: the extraction layer reads from CAT-SOOP's on-disk data root (ideally on a schedule, in batch), and the rest of the pipeline operates entirely on the copy it produces. This means the analytics system can be developed, tested, and iterated on without touching CAT-SOOP's own codebase, and a bug in the analytics pipeline cannot affect the live course.

Design decision: we do not assume PostgreSQL, or any specific database, up front. The first task is to determine what CAT-SOOP's own storage actually looks like (Sections 9 and 13); the analytics database is chosen to fit that reality, not the other way around.

6. Functional Requirements

ID	Requirement	Description
FR1	Course-level overview	Show total/active students, overall completion, average scores, and module progression for the whole course.
FR2	Module-level progress	Show, per module, how many students reached it, how many completed it, and completion percentage.
FR3	Student-level progress	Show, per student, modules reached/completed, scores, attempts, and recent activity.
FR4	Assessment analytics	Show per-assessment score averages, distributions, and completion rates.
FR5	Attempt analytics	Show attempt counts per student per question/exercise, and attempt-count distributions.
FR6	Score analytics	Show score distributions at module and question level, and how scores compare to class averages.
FR7	Module difficulty analysis	Surface a heuristic difficulty indicator per module (Section 11).
FR8	Question/exercise difficulty analysis	Surface a heuristic difficulty indicator per question (Section 11).
FR9	Progress-over-time analysis	Show how class-wide and individual progress changes across the term, using daily snapshots.
FR10	At-risk student identification	Flag students whose activity, completion, or scores fall notably behind the cohort.
FR11	Filtering and drill-down	Allow the instructor to move from course -> module -> question -> student views.

ID	Requirement	Description
FR12	Historical analytics	Retain daily snapshots so trends can be viewed over time, not just as a current snapshot.
FR13	Instructor-only access	Restrict the dashboard to authenticated instructor/admin accounts (Section 19).
FR14	Batch data synchronization	Run extraction and metric computation on a daily schedule, matching the professor's stated requirement.
FR15	Multi-course support (design-time)	Keep the schema course-scoped so a second course can be added without redesign, even if only one course is deployed initially.
FR16	Scalability	Architect for growth from ~250 to a public-course scale, evaluated rather than assumed (Section 20).

7. Inputs

- CAT-SOOP course structure (modules, pages, exercises, as defined in course content files).
- Student/user information as CAT-SOOP records it (username/ID and role: student, guest, admin).
- Exercise/question definitions and their auto-grading (checker) configuration.
- Assessment submissions and checker/autograder results.
- Attempt records and their timestamps, to the extent CAT-SOOP's log format exposes them.
- Scores, as computed and stored by CAT-SOOP's own grading logic.
- Any page/module access events, if present in the log store (needs verification — see Section 12.3).

Optional / future input: Google Classroom submission metadata for the video component (submission timestamp, link). This is explicitly not a core requirement (Section 17, MVP scope)..

8. Outputs

8.1 Course level

- Total and active student counts
- Overall completion percentage and average scores
- Module-by-module progression summary
- Count of students behind schedule
- Overall assessment statistics

8.2 Module level

- Students who reached / completed the module, and completion percentage
- Average score and score distribution
- Number of attempts and failure rate
- Activity trend over time; comparison across modules

8.3 Question / assessment level

- Attempt count, average score, first-attempt score, best score
- Failure rate and count of struggling students
- Score distribution and a heuristic difficulty indicator

8.4 Student level

- Modules reached / completed; scores; attempts; recent activity
- Progress-over-time view and weak-area identification
- Comparison to course/cohort averages

9. Proposed Dashboard

The dashboard is organized as a small set of pages, each answering specific, named instructor questions rather than presenting data for its own sake.

Page	Instructor Question It Answers	Key Content
Course Overview	"How is the class doing overall?"	KPI cards (active students, completion %, average score), module-progression bar chart.
Module Analytics	"Which modules are students stuck on?"	Per-module completion bars, attempt/failure-rate table, drop-off chart across module sequence.
Assessment / Question Analytics	"Which exercises are hardest?"	Score-distribution histograms, attempts-vs-score scatter, difficulty ranking table.
Student Analytics	"How is this specific student doing?"	Per-student progress timeline, score table, comparison to cohort average.
Progress Trends	"Is the class keeping pace over time?"	Line chart of cumulative completion / activity using daily snapshots.
Students Requiring Attention	"Who should I check in on?"	Ranked list built from the at-risk heuristic in Section 11.
Comparison / Insights	"What stands out this week?"	Module-to-module and cohort comparisons, surfaced as short auto-generated observations.

10. Proposed Visualizations

Visualization	Question It Answers
KPI cards	"Where do we stand right now?" (single numbers: completion %, avg score, active students)
Bar chart (per module / per question)	"How does X compare across modules/questions?"
Line chart (time series)	"Is progress on track over the term?"

Visualization	Question It Answers
Histogram	“What does the score/attempt distribution look like?”
Student x Module heatmap / matrix	“Where exactly are students stuck, across the whole class at once?”
Progress bars (per student / per module)	“How close is this student/module to completion?”
Attempt-count distribution	“Are students needing many attempts on a specific question?”
Cohort progression chart	“Is this student ahead of, on par with, or behind the cohort?”

Charts are added only where they answer one of the instructor questions above; the dashboard deliberately avoids purely decorative visualizations.

11. Difficulty and Progress Indicators

Framing: the indicators below are proposed heuristic metrics for an initial, transparent, interpretable dashboard — not scientifically validated learning-analytics models. They are simple, explainable aggregates the instructor can sanity-check against what he already knows about the class.

11.1 Module difficulty (proposed heuristic)

- Average score on the module's assessments
- Failure rate and average number of attempts
- Completion rate and drop-off rate relative to the previous module

11.2 Assessment/question difficulty (proposed heuristic)

- Average score and first-attempt success rate
- Number of attempts needed and percentage of students failing
- Shape of the score distribution (e.g., bimodal vs. concentrated)

11.3 Student progress / at-risk flag (proposed heuristic)

- Completion relative to the cohort and recency of last activity
- Assessment performance and attempt patterns
- Progress trend relative to the cohort's trend, not just an absolute number

12. Data Architecture — CAT-SOOP Storage Investigation

This section documents what has actually been confirmed about CAT-SOOP's storage and logging mechanism from its own documentation and public source history, as the professor specifically asked us to verify rather than assume this.

12.1 Confirmed from CAT-SOOP's own documentation and source history

Aspect	What we found	Source
Storage mechanism	File-based, not a conventional SQL/NoSQL application database by default. CAT-SOOP's own installation documentation states: “all of CAT-SOOP's data are stored in files on disk in a directory called <code>_logs</code> ” inside the configured <code>cs_data_root</code> (default <code>~/local/share/catsoop</code>).	catsoop.org install/server-configuration docs
Logging module	The logging subsystem lives in <code>catsoop/cslog.py</code> (renamed from an earlier <code>logging.py</code> specifically to avoid clashing with Python's built-in logging module — confirming it is CAT-SOOP's own bespoke logging layer, not a wrapper around a standard DB driver).	CAT-SOOP CHANGELOG
Format history	The on-disk log format has changed over CAT-SOOP's history: an original custom “ <code>catsoopdb</code> ” format, a period where SQLite was used instead, and later commits (“change log files to be plain-text”) moving log files toward a plain-text, human-readable representation.	CAT-SOOP CHANGELOG; catsoop.org/git commit history
Optional protections	CAT-SOOP supports optional encryption (<code>cs_log_encryption_passphrase</code> / <code>cs_log_encryption_salt</code> , implemented via a <code>fernet.py</code> module) and optional compression (<code>cs_log_compression</code>) of log entries. Public-facing installations are explicitly advised to encrypt the logs directory.	CAT-SOOP CHANGELOG; catsoop.org/git commit history; install docs
Log tooling	CAT-SOOP ships CLI utilities — <code>logread</code> , <code>logwrite</code> , <code>logedit</code> — for inspecting and editing individual log entries in human-readable form, confirming log entries are discrete, addressable records rather than an opaque blob.	CAT-SOOP CHANGELOG
Alternate backend (needs re-verification)	At least one historical version of the README documents an optional MongoDB-backed mode, enabled via environment variables (<code>USE_CLOUD_DB=mongodb</code> , <code>MONGODB=<connection string></code>). Whether this is present, maintained, or relevant in the version the professor runs is unconfirmed.	README, personal GitHub mirror of catsoop (ichuang/catsoop)

12.2 A note on the reference repository the professor supplied

The GitHub link provided in the problem statement (github.com/ichuang/catsoop) is a personal mirror/fork last touched around 2018–2019, and its own file statistics (predominantly JavaScript, a small fraction Python) do not match the shape of a Python LMS codebase — suggesting it may be a partial or stale mirror, not the canonical source. CAT-SOOP's own README states its authoritative source is hosted on a self-run Gitea/Forgejo instance at catsoop.org/git/catsoop/catsoop (mirrored at catsoop.mit.edu/git). We used this canonical source, plus CAT-SOOP's own published documentation, for the technical claims in this section, and flag this discrepancy explicitly rather than silently treating the GitHub mirror as authoritative.

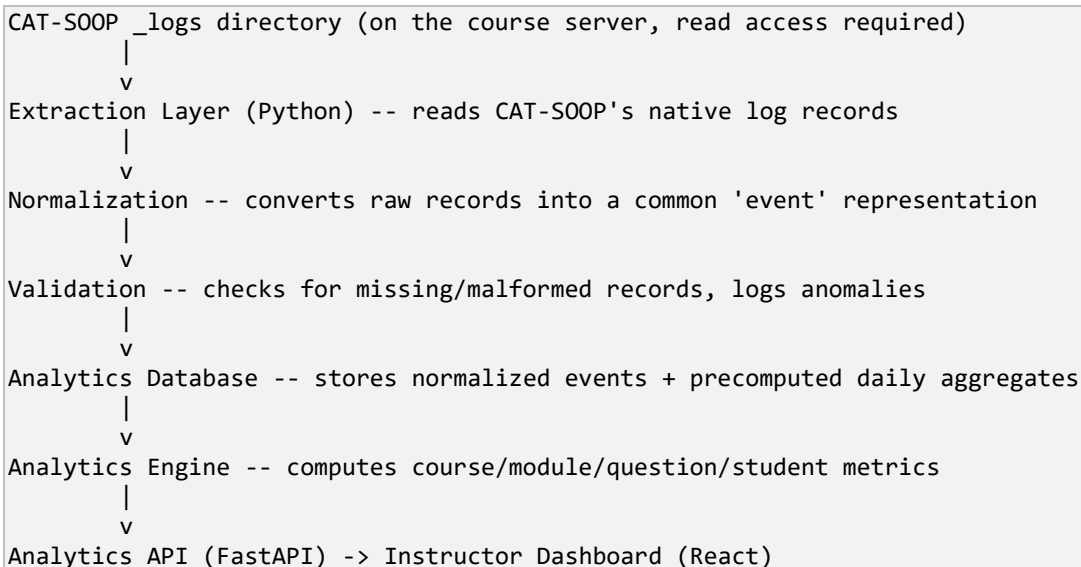
Also confirmed directly from CAT-SOOP's maintainer: the official documentation site states its own docs are “extremely sparse right now.” This is consistent with our approach of prioritizing source-code and changelog evidence over documentation, and of verifying uncertain points experimentally against a local instance rather than assuming behavior.

12.3 Needs verification experimentally (local CAT-SOOP instance)

- The exact on-disk log record schema for the CAT-SOOP version actually deployed at IITGN (version may differ from the current catsoop.org/git head).
- Whether page/module **view** events are logged at all, or whether only submission/attempt events are — this directly determines whether “reached a module” can be measured directly or must be approximated (e.g., as “first attempt recorded on any exercise in that module”).
- The exact semantics of an “attempt” (e.g., does every checker invocation count, including page reloads or accidental submissions?).
- Whether the deployed instance uses the plain-text log format, an older SQLite-backed format, or encryption/compression — this affects how the extraction layer reads the files.
- Whether CAT-SOOP already exposes any reusable statistics/reporting functions or internal APIs we can call instead of parsing log files directly (to be checked once we have local source access to the exact deployed version).

Wherever a technical claim in this document could not be confirmed from the source/documentation evidence above, it is marked “needs to be verified experimentally” rather than assumed.

12.4 Proposed extraction and processing pipeline



A separate analytics store, rather than querying CAT-SOOP's raw files on every dashboard request, is proposed because: (a) CAT-SOOP's files are not indexed for the kind of aggregate queries a dashboard needs, so repeated raw parsing would be slow and would grow slower as the course grows; (b) it keeps the analytics workload off the live CAT-SOOP process entirely, which matters since CAT-SOOP is also handling live student traffic; and (c) it lets us keep dated snapshots for the progress-over-time views (FR9, FR12) that raw CAT-SOOP logs may not retain in an easily queryable form.

13. Proposed Analytics Database Schema (Conceptual)

Important distinction: this is our proposed analytics schema, not CAT-SOOP's native data model. CAT-SOOP's native model is whatever Section 12 determines it to be. This schema is what we build downstream of extraction, and will be finalized only after inspecting real CAT-SOOP log data locally.

```
Course
|-- Module
|   |-- Exercise / Assessment
|       |-- Attempt (student, timestamp, score, attempt_number)
|-- Student
    |-- Activity (student, page/module, timestamp, event_type)
    |-- Score (student, exercise, score, recorded_at)
    |-- DailySnapshot (student or course, date, precomputed metrics)
```

Entity	Purpose
Course	One row per CAT-SOOP course tracked; keeps the schema multi-course-ready (FR15).
Module	A content grouping within a course, as CAT-SOOP's course structure defines it.
Exercise / Assessment	An individual auto-graded question/exercise within a module.
Student	A CAT-SOOP user with the student role, scoped to a course.
Attempt	One recorded submission/check against an exercise, with score and timestamp.
Activity	Any other loggable event (e.g., a page visit, if the data supports it) used to infer "reached."
Score	The current/best recorded score for a student on an exercise, derived from Attempts.
DailySnapshot	A precomputed, dated rollup of course/module/student metrics, enabling FR9 and FR12 without recomputation from raw events each time.

14. Data Pipeline

1. Access CAT-SOOP's data root on the server (read access to _logs and course content).
2. Extract raw log records for the relevant course.
3. Parse records into a common event structure (attempt, score, activity).
4. Normalize records against the course/module/exercise structure.
5. Validate: flag missing, duplicate, or malformed records rather than silently dropping them.
6. Store normalized events and updated aggregates in the analytics database.
7. Compute course/module/question/student metrics.
8. Update the dashboard's data (via the API layer, not by re-parsing CAT-SOOP on each page view).

The professor's stated requirement is batch processing **once per day**. This is the initial target for the pipeline scheduler. The extraction/normalization steps are written to be idempotent and incremental (only new records since the last run are processed) so that, if needed later, the same pipeline can be run more frequently without

redesign — this is an architectural property to design in now, not a commitment to build a real-time system in this project.

15. Local CAT-SOOP Experimental Environment

Before touching the real IITGN deployment, all data-format investigation happens against a local, disposable CAT-SOOP instance, per the professor's explicit instruction. CAT-SOOP publishes a `sample_course` repository (catsoop.org/catsoop/sample_course) intended for exactly this kind of local bring-up.

15.1 Setup steps

9. Clone the canonical CAT-SOOP source (catsoop.org/git/catsoop/catsoop) and install locally per the official install docs.
10. Run ``catsoop configure`` and ``catsoop start``; confirm the local instance serves the default page.
11. Clone/adapt the official `sample_course` into the local data root as a stand-in for a Digital-Systems-style course.
12. Create several dummy modules with embedded auto-graded exercises.
13. Register multiple dummy student accounts.
14. Script each dummy persona's behavior through the CAT-SOOP web interface (or a browser-automation harness).
15. Inspect the resulting contents of the `_logs` directory: file layout, naming, and record format.
16. Map specific dummy actions to specific on-disk records (e.g., “Student A submitted exercise 2 for the third time” -> which file, which record).
17. Build and iterate the extraction-layer prototype against this known, controlled dataset.
18. Validate computed analytics against the known scripted behavior before pointing the pipeline at any real course data.

15.2 Synthetic student personas

Persona	Behavior
Student A	Completes every module; consistently high scores; few attempts per exercise.
Student B	Repeated attempts on most exercises; scores improve gradually across attempts.
Student C	Active and completing modules early on, then stops entirely after Module 2.
Student D	Generally on pace, but performs poorly on one specific assessment (many failed attempts).
Student E	Completes modules on schedule but with consistently low scores.

These personas give the analytics system a ground truth: if Student C is not flagged as having stalled after Module 2, or Student D's poor performance on one specific assessment is not surfaced, the corresponding metric is wrong and must be fixed before any real data is processed.

16. Development Methodology and Phases

The project follows an iterative, incremental methodology: each phase produces a working, testable artifact, and later phases build on validated earlier ones rather than assuming untested groundwork.

Phase	Name	Objective / Tasks	Output / Deliverable
1	CAT-SOOP research	Read source/docs; document confirmed vs. unverified storage and logging behavior (Section 12).	Research findings memo (this document, Section 12, is the first version).
2	Local deployment	Stand up a local CAT-SOOP instance with a test course.	Running local instance.
3	Data exploration	Create dummy students/personas; generate activity; inspect resulting log files.	Data-exploration report mapping actions to on-disk records.
4	Extraction prototype	Build a script that reads local log files into a structured, in-memory representation.	Extraction prototype, validated against known persona actions.
5	Analytics database	Finalize and implement the analytics schema (Section 13); build the load path.	Populated analytics database from the local test course.
6	Analytics engine	Implement course/module/question/student metric computations (FR1–FR9).	Metric computation module with unit tests against synthetic data.
7	Dashboard MVP	Build the API layer and the core dashboard pages (Course Overview, Module, Student).	Working MVP dashboard against local test data.
8	Advanced analytics	Difficulty indicators, at-risk flagging, trend views (FR7, FR8, FR10).	Extended dashboard pages.
9	Integration with real deployment	Point extraction at the IITGN CAT-SOOP instance (with appropriate access/permission), re-validate assumptions from Section 12.3.	Pipeline running against real course data (read-only).
10	Testing and scalability evaluation	Correctness, load, and scale testing (Sections 18, 16).	Test report; documented scalability findings.

17. MVP vs. Advanced Features

17.1 MVP

- Course overview page
- Module-level progress view
- Student-level progress view
- Scores and attempts, per module and per exercise
- Basic assessment statistics
- Basic filtering (by module, by student)
- Daily batch data update

- A functioning, if simple, dashboard front end

17.2 Advanced (post-MVP, time and data permitting)

- Difficulty indicators (module and question level)
- At-risk-student identification
- Time-series / progress-trend analytics
- Cohort comparisons
- Deeper question-level analysis
- Anomaly detection
- Predictive analytics
- Google Classroom / video-submission integration
- More-frequent-than-daily updates
- Full multi-course support in the deployed system (schema already supports it — Section 13)

18. Technical Stack

Layer	Technology	Why
Extraction / ETL	Python	CAT-SOOP itself is Python-based, so a Python extraction layer can, if useful, reuse CAT-SOOP's own parsing utilities and runs naturally alongside it operationally.
Analytics database	Relational DB (engine TBD, e.g. PostgreSQL or SQLite) — chosen after Phase 1–3 findings	We will not commit to a specific engine until Section 12's investigation is complete; the schema in Section 13 is designed to be portable across relational engines.
Analytics API	Python + FastAPI	Straightforward to build a typed REST API over the analytics database; keeps the same language as the extraction layer, reducing context-switching for a student team.
Dashboard front end	React + a charting library (e.g., a standard React chart package)	Component-based UI suits the multi-page, drill-down dashboard described in Section 9.
Scheduler	Cron (initial) / a lightweight task queue if incremental processing needs grow	Matches the stated once-daily batch requirement; a heavier scheduler is not justified at this scale yet.
Deployment	Docker, if the target server permits it	Simplifies moving the analytics stack between the local test environment and the deployment environment.
Testing	pytest (backend); a standard front-end testing library	Matches the chosen backend/frontend languages.

19. Security and Access Control

- Only instructor/admin accounts can access the analytics dashboard (FR13); students are never given dashboard access in this design.
- Credentials or filesystem access needed to read CAT-SOOP's data root are never hardcoded; they are supplied via configuration/secrets management, consistent with CAT-SOOP's own recommendation that a public-facing log directory be encrypted.
- The analytics database and API are not exposed on the public internet without authentication.
- Student-level data is only surfaced to the instructor role; aggregate/course-level views are preferred wherever individual identification is not required.
- Role-based access control is enforced at the API layer, not just hidden in the front end.

20. Scalability

The professor is considering making the course public, so the system should not be designed narrowly around 250 students. This project does not claim a specific number of supportable users up front; instead, scalability is treated as something to design for and then evaluate.

Consideration	How it helps
Separate analytics database	Keeps heavy aggregate queries off CAT-SOOP's own live process entirely.
Indexed queries / precomputed aggregates	Avoids recomputing course-wide metrics from raw events on every dashboard load.
Daily snapshots	Bounds the amount of historical data that needs to be scanned for trend views.
Pagination and caching	Keeps dashboard pages responsive as student counts grow.
Asynchronous / incremental ingestion	Extraction only processes new records since the last run, so pipeline time grows with new activity, not total history.
Stateless API layer	Allows the API to be scaled horizontally if needed.
Containerized deployment	Makes it straightforward to move or replicate the analytics stack.

Whether the resulting architecture actually supports 1,000, 5,000, or 10,000+ students will be assessed through the load-testing activity in Phase 10 (Section 16), not asserted in advance.

21. Testing Strategy

- Unit testing of extraction, normalization, and metric-computation logic.
- Integration testing of the full pipeline: CAT-SOOP log files in, dashboard-ready metrics out.
- Data-validation testing: malformed or missing records are flagged, not silently ignored.
- Dashboard testing: each page renders correctly against known test data.
- Authentication/authorization testing: non-instructor accounts cannot reach dashboard endpoints.
- Performance and load testing at increasing simulated student counts (Section 20).

- Regression testing as the extraction layer is adapted from the local test format to the real deployment's format.

21.1 Correctness validation against synthetic personas

- If Student B makes exactly N attempts on an exercise, the dashboard must report exactly N attempts — not more, not fewer.
- If Student C never accesses Module 4, the dashboard must not report Module 4 as reached for that student.
- If Student D fails a specific assessment repeatedly, that assessment must appear with a correspondingly high failure rate / low first-attempt score.

22. Risks and Challenges

Risk	Mitigation
CAT-SOOP's on-disk log format is not fully documented	Prioritize source-code and local-instance evidence over documentation; verify experimentally (Section 12.3) before building on any assumption.
IITGN's deployed CAT-SOOP version may differ from the current canonical source	Confirm the deployed version early (Phase 9) and re-run the local-instance validation against a matching version if a mismatch is found.
Required student activity may not be logged at all (e.g., page visits)	Define an alternative, explicitly-approximate interpretation of “reached a module” (e.g., first attempt within it) if direct page-view data is unavailable; document the approximation clearly in the dashboard itself.
Attempt semantics may be ambiguous (e.g., reloads counted as attempts)	Confirm exact semantics against the local test instance using controlled, scripted actions (Section 15) before trusting attempt counts.
CAT-SOOP's internal functions/APIs are not designed as a public interface	Prefer reading the log store directly over depending on internal functions that could change without notice; treat any internal API use as an optimization, not a dependency.
Large courses could produce large volumes of log data	Use incremental, batch extraction (Section 14) rather than full re-scans; evaluate performance at scale (Section 20).
Read access to the production CAT-SOOP data root may be restricted for good reason (student privacy)	Coordinate access scope explicitly with the professor/IITGN IT before Phase 9; keep the analytics pipeline strictly read-only against CAT-SOOP's data.
Changes to CAT-SOOP's on-disk format between versions could break the extractor	Isolate all format-specific parsing in one extraction module so future format changes require a localized fix, not a redesign.

23. Assumptions and Open Questions

23.1 Known

- The Digital Systems course has approximately 250 students.
- CAT-SOOP already handles most course content and auto-graded assessments.
- Google Classroom is used primarily for YouTube/video submission, not for content or grading.
- The main missing capability is instructor-facing analytics, not LMS functionality.
- The professor wants course-, module-, student-, and assessment-level analytics.
- Future scalability to a public course is a stated design goal.
- CAT-SOOP stores its data in files on disk under a configurable data root, per its own documentation (Section 12.1).

23.2 Needs verification

- The exact CAT-SOOP version running at IITGN.
- The exact storage/log format in use on that deployment (plain-text, SQLite-backed, encrypted, compressed).
- The exact structure and completeness of the available logs (Section 12.3).
- Whether page/module-visit events are recorded at all.
- The exact semantics of an “attempt.”
- Whether CAT-SOOP exposes any reusable internal statistics functions we can call directly.
- How much, if any, modification to CAT-SOOP itself will be necessary versus purely external extraction.
- The final deployment environment for the analytics stack (co-located with CAT-SOOP, or a separate server with read access to its data root).

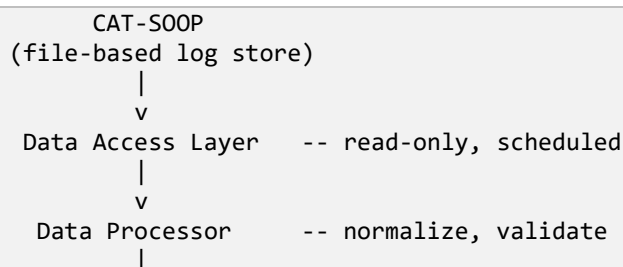
24. Expected Deliverables

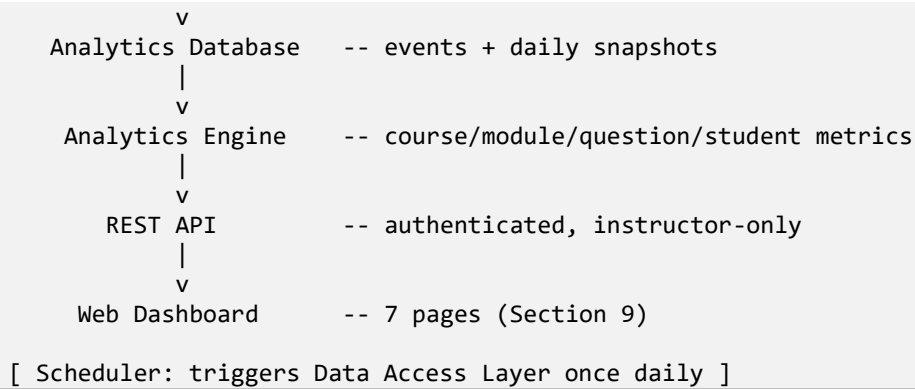
19. Local CAT-SOOP development environment
20. Dummy Digital-Systems-style test course
21. Synthetic student dataset (personas, Section 15.2)
22. CAT-SOOP data-exploration report (confirmed log structure, mapped to persona actions)
23. Data extraction module
24. Analytics database schema and implementation
25. Analytics processing pipeline (daily batch)
26. Analytics API
27. Instructor dashboard (MVP, plus advanced features as time allows)
28. Automated daily data-processing job
29. Testing suite (unit, integration, validation)
30. Documentation (setup, architecture, schema, known limitations)
31. Deployment instructions
32. Final demonstration against both the local test course and, if access permits, the real IITGN course

25. Success Criteria

- The system ingests data from a test CAT-SOOP course end to end.
- The dashboard correctly represents every synthetic test student's behavior (Section 21.1).
- Module completion percentages, attempt counts, and scores match CAT-SOOP's own records exactly for the test course.
- The instructor can drill down: course -> module -> assessment -> student.
- The daily synchronization pipeline runs reliably without manual intervention.
- The dashboard remains usable (not just functional, but responsive) as simulated student counts grow into the hundreds/low thousands.
- Every analytic shown has been validated against a known, scripted ground truth before being trusted on real course data.

26. Proposed Final Architecture





27. Project Timeline

Milestones are used in place of fixed dates, since some durations (particularly Phase 1–3 investigation) depend on how quickly CAT-SOOP's real log structure can be understood.

Milestone	Description	Corresponding Phase(s)
M1	CAT-SOOP research + local setup	Phases 1–2
M2	Data extraction validated against synthetic personas	Phases 3–4
M3	Analytics database + ETL pipeline running end to end	Phase 5
M4	Basic analytics (FR1–FR6) computed correctly	Phase 6
M5	Dashboard MVP live against local test data	Phase 7
M6	Advanced analytics added (FR7, FR8, FR10)	Phase 8
M7	Integrated with real deployment (where access permits); tested at scale	Phases 9–10
M8	Deployment package + documentation finalized	Phase 10

28. Final Summary

CAT-SOOP already does the hard work of delivering course content and auto-grading exercises for the Digital Systems course. What it lacks, in this deployment, is a way for the instructor to see that activity in aggregate. This project builds that missing analytics layer: a read-only extraction pipeline over CAT-SOOP's file-based log store, a purpose-built analytics database, and an instructor dashboard covering course, module, question, and student views.

The architecture is deliberately decoupled from CAT-SOOP's own request path, so it can be developed and validated independently and scaled up separately as the course potentially grows into a public offering. Every technical claim about CAT-SOOP's internals in this document is grounded in its own documentation or public source history, and every remaining unknown is named explicitly rather than assumed.

The first technical milestone is not dashboard code — it is standing up a local CAT-SOOP instance, generating known synthetic student activity, and confirming exactly what that activity looks like once it lands in `cs_data_root/_logs`. Everything else in this plan is built on top of that finding.